

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
КАФЕДРА ИНФОРМАТИКИ

Отчёт по лабораторной работе №7

По теме «Криптография с использованием эллиптических кривых»

Выполнил:
студент гр. 153501
Власенко Т. П.

Проверил:
Герчик А.В.

Минск 2024

СОДЕРЖАНИЕ

Введение.....	3
1 Теоретические сведения	4
2 Результат выполнения задачи	6
Заключение	7
Приложение А	8

ВВЕДЕНИЕ

Реализовать схему шифрования (дешифрования) для аналога алгоритма Эль-Гамала на основе эллиптических кривых. Написать функцию для генерации открытого и закрытого ключей с использованием эллиптических кривых. Реализовать алгоритм для генерации параметров эллиптической кривой, включая выбор простого числа и коэффициентов.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Схема Эль-Гамала (Elgamal) — криптосистема с открытым ключом, основанная на трудности вычисления дискретных логарифмов в конечном поле. Криптосистема включает в себя алгоритм шифрования и алгоритм цифровой подписи. Схема Эль-Гамала лежит в основе бывших стандартов электронной цифровой подписи в США (DSA) и России (ГОСТ Р 34.10-94).

Генерация ключей:

- 1 Генерируется случайное простое число p .
- 2 Выбирается целое число g — первообразный корень p .
- 3 Выбирается случайное целое число x такое, что $(1 < x < p - 1)$.
- 4 Вычисляется $y = g^x \bmod p$.
- 5 Открытым ключом является (y, g, p) , закрытым — число x .

Шифрование

Сообщение должно быть меньше числа p . Сообщение шифруется следующим образом:

- 1 Выбирается сессионный ключ — случайное целое число, k такое, что $1 < k < p - 1$.
- 2 Вычисляются числа $a = g^k \bmod p$ и $b = y^k M \bmod p$.
- 3 Пара чисел (a, b) является шифротекстом.

Расшифрование

Зная закрытый ключ x , исходное сообщение можно вычислить из шифротекста (a, b) по формуле: $M = b(a^x)^{-1} = ba^{p-1-x} \bmod p$

Схема шифрования представлена на рисунке ниже:

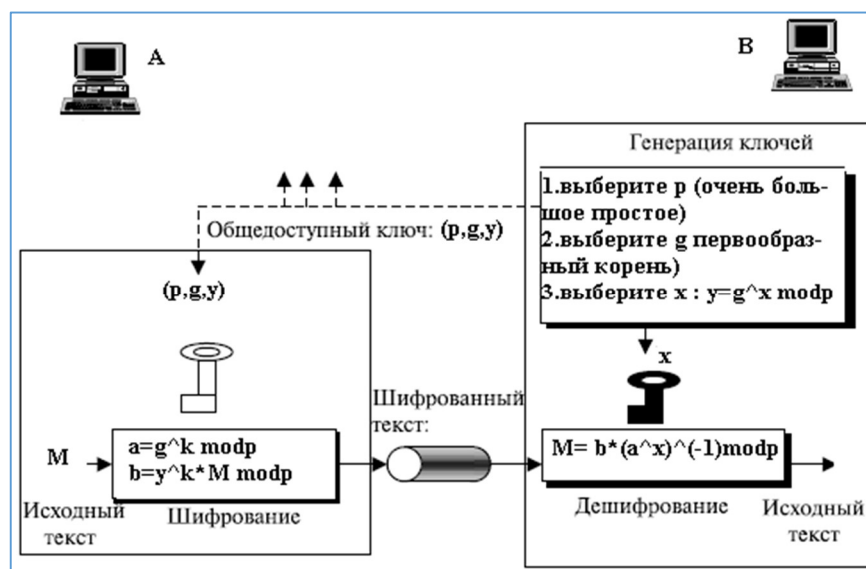


Рисунок 1 – Схема шифрования

2 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ЗАДАЧИ

В результате выполнения лабораторной работы была реализована схема шифрования (дешифрования) для аналога алгоритма Эль-Гамала на основе эллиптических кривых. Написана функция для генерации открытого и закрытого ключей с использованием эллиптических кривых. Реализован алгоритм для генерации параметров эллиптической кривой, включая выбор простого числа и коэффициентов. Результаты работы программы можно видеть на рисунке 2:

[illegible]

Рисунок 2 – Результат выполнения программы

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной лабораторной работы была реализована схема шифрования (дешифрования) для аналога алгоритма Эль-Гамала на основе эллиптических кривых. Написана функция для генерации открытого и закрытого ключей с использованием эллиптических кривых. Реализован алгоритм для генерации параметров эллиптической кривой, включая выбор простого числа и коэффициентов.

ПРИЛОЖЕНИЕ А

(обязательное)

Код программы

main.py

```
import random
import binascii

# Параметры эллиптической кривой
p = 2 ** 256 - 2 ** 32 - 2 ** 9 - 2 ** 8 - 2 ** 7 - 2 ** 6 - 2 ** 4 - 1
a = 0
b = 7
G = (0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798,
     0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8)
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        if other is None:
            return self

        if self.x == other.x and self.y == other.y:
            lambd = (3 * self.x ** 2 + a) * pow(2 * self.y, p - 2, p) % p
        else:
            lambd = (other.y - self.y) * pow(other.x - self.x, p - 2, p) % p

        x = (lambd ** 2 - self.x - other.x) % p
        y = (lambd * (self.x - x) - self.y) % p
        return Point(x, y)

    def __mul__(self, n):
        if n == 0:
            return None

        Q = self
        R = None
        while n:
            if n & 1:
                R = R + Q if R else Q
            Q = Q + Q
            n //= 2
        return R

def generate_keys():
    private_key = random.randint(1, n - 1)
    public_key = Point(*G) * private_key
    return private_key, public_key

def encrypt(message, public_key):
    encrypted = b''
    k = random.randint(1, n - 1)
    R = Point(*G) * k
    for byte in message:
        m = Point(byte, 0)
        S = public_key * k
```

```

        c_x = (m.x + S.x) % p
        c_y = (m.y + S.y) % p
        encrypted += c_x.to_bytes(32, byteorder='big') + c_y.to_bytes(32,
byteorder='big')
        return R, encrypted

# Дешифрование
def decrypt(R, encrypted, private_key):
    decrypted = b''
    for i in range(0, len(encrypted), 64):
        c_x = int.from_bytes(encrypted[i:i + 32], byteorder='big')
        c_y = int.from_bytes(encrypted[i + 32:i + 64], byteorder='big')
        c = Point(c_x, c_y)
        S = R * private_key
        m_x = (c.x - S.x) % p
        m_y = (c.y - S.y) % p
        m = Point(m_x, m_y)
        decrypted += bytes([m.x])
    return decrypted.decode('utf-8')

# Пример использования
plaintext = ('Привет, мир! Hello world 1234 !@#$ Добрый вечер, добрый день
вечер, добрый день  вечер, добрый день Добрый вечер, '
            'добрый день  вечер, добрый день  вечер, добрый день ')
private_key, public_key = generate_keys()
R, ciphertext = encrypt(plaintext.encode('utf-8'), public_key)
decrypted = decrypt(R, ciphertext, private_key)
print(f'Открытый ключ: {public_key.x}, {public_key.y}')
print(f'Закрытый ключ: {private_key}')
print(
    f'Зашифрованное сообщение: {binascii.hexlify(R.x.to_bytes(32,
byteorder="big") + R.y.to_bytes(32, byteorder="big") + ciphertext)}')
print(f'Расшифрованное сообщение: {decrypted}')

```